Online on xv6 System Call

Section: A1 — **Time:** 35 minutes

In this assignment, you will let a process reroute its own system calls. A small table inside struct proc maps one system call number to another, and the kernel consults that table on every system call the process makes, running the handler of the mapped number instead. A user space program will fetch the table using kernel space memory copying.

Required System Calls:

1. `redirectSyscall(int from, int to)` : Installs a mapping so that, from this point on, whenever the calling process makes system call number from, the kernel runs the handler of system call number to instead and returns its result. Returns 0 on success, and -1 if the table is already full or either number is not a valid system call.
2. `getRedirects(struct redirect_table *rt)` : Copies the calling process's active mappings into the provided user space struct pointer. Returns 0 on success and -1 on invalid memory address.

The structure definitions are as follows:

```
#define NREDIRECT 3

struct redirect_entry {
    int from;    // syscall number the process asks for
    int to;      // syscall number actually run
};

struct redirect_table {
    int count;                          // active mappings (max NREDIRECT)
    struct redirect_entry entries[NREDIRECT];
};
```

Your implementation must satisfy all of the following:

- A process may hold at most `NREDIRECT` mappings at a time. Once the table is full, a further `redirectSyscall` must fail and leave the existing mappings untouched.
- If a mapping for `from` already exists, it is replaced rather than added, and the count does not change.
- **A mapping must never be followed more than once.** If 14 maps to 11 and 11 maps to 13, then calling 14 must run the handler of 11, not of 13.
- The mappings are stored in the order they were first installed, and `getRedirects` reports them in that order.
- The table is per process. A process that has installed no mapping must behave exactly like unmodified xv6, and a newly created process must never pick up a table left behind by a process that has already exited.

Mechanics:

- Update `syscall.c` and `struct proc` so that, before dispatching, the kernel looks the system call number up in the calling process's table and runs the handler of the mapped number instead.
- Reject a `from` or a `to` that is not a valid system call number, and never index the handler table with a number that has no handler. Doing so would jump through an empty slot and panic the kernel.
- **While testing, redirect only system calls that take no arguments**, such as `getpid` (11) and `uptime` (14). Redirecting `write` (16) or `exit` (2) will break your own program, since it needs those to print and to terminate.

Required User Program:

`redir from to [from to ...]` , which installs each pair in turn and reports any that fail, then prints uptime() and getpid(), and finally prints the table.

The body of the program is given below.

```
int
main(int argc, char *argv[])
{
    struct redirect_table rt;
    int i;

    if(argc < 3 || (argc % 2) == 0){
        fprintf(2, "Usage: %s from to [from to ...]\n", argv[0]);
        exit(1);
    }

    for(i = 1; i + 1 < argc; i += 2){
        if(redirectSyscall(atoi(argv[i]), atoi(argv[i+1])) < 0)
            printf("redirectSyscall(%d, %d) failed\n",
                   atoi(argv[i]), atoi(argv[i+1]));
    }

    printf("uptime() = %d\n", uptime());
    printf("getpid() = %d\n", getpid());

    if(getRedirects(&rt) < 0){
        fprintf(2, "getRedirects failed\n");
        exit(1);
    }

    printf("-- Redirects --\n");
    printf("count: %d\n", rt.count);
    for(i = 0; i < rt.count; i++)
        printf("%d -> %d\n", rt.entries[i].from, rt.entries[i].to);

    exit(0);
}
```

See the sample I/O for clarification:

```
$ redir 14 11
uptime() = 3
getpid() = 3
-- Redirects --
count: 1
14 -> 11
$ redir 14 11 11 14
```

```
uptime() = 4
getpid() = 48
-- Redirects --
count: 2
14 -> 11
11 -> 14
$ redir 14 13 14 11
uptime() = 5
getpid() = 5
-- Redirects --
count: 1
14 -> 11
$ redir 14 11 11 14 8 11 9 11
redirectSyscall(9, 11) failed
uptime() = 6
getpid() = 78
-- Redirects --
count: 3
14 -> 11
11 -> 14
8 -> 11
```

Explanation:

- 14 is the system call number of `uptime` and 11 is that of `getpid`.
- In the first command `uptime` has been rerouted to `getpid`, so `uptime`() returns the pid and both lines print the same number.
- The second command installs two mappings at once, each pointing at the other. `uptime()` runs the handler of `getpid` and prints the pid, while `getpid()` runs the handler of `uptime` and prints a tick count. Neither is followed twice: a mapping is applied once and then the call is made. If mappings were followed repeatedly, `uptime()` would be sent straight back to `uptime` again.
- In the third command both pairs have `from = 14`, so the second one replaces the first instead of being added, and the count stays 1.
- In the fourth command the table is already full when the fourth pair arrives, so that one is refused and the first three remain in place.
- The tick counts (48 and 78 above) depend on how long xv6 has been running, and the pids depend on how many programs you have run, so those numbers will differ on your machine. The structure of the output must be the same.

Submission:

```
git add --all
git diff HEAD > ../{studentID}.patch
```